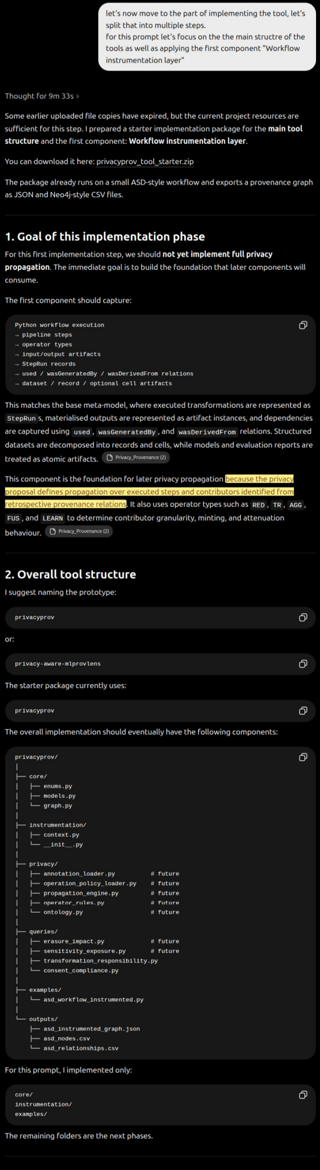

let's now move to the part of implementing the tool, let's split that into multiple steps.
for this prompt let's focus on the the main structre of the tools as well as applying the first component "Workflow instrumentation layer"

Thought for 9m 33s ›

Some earlier uploaded file copies have expired, but the current project resources are sufficient for this step. I prepared a starter implementation package for the **main tool structure** and the first component: **Workflow instrumentation layer**.

You can download it here: privacyprov_tool_starter.zip

The package already runs on a small ASD-style workflow and exports a provenance graph as JSON and Neo4j-style CSV files.

---

1. Goal of this implementation phase

For this first implementation step, we should **not yet implement full privacy propagation**. The immediate goal is to build the foundation that later components will consume.

The first component should capture:

```
Python workflow execution
→ pipeline steps
→ operator types
→ input/output artifacts
→ StepRun records
→ used / wasGeneratedBy / wasDerivedFrom relations
→ dataset / record / optional cell artifacts
```

This matches the base meta-model, where executed transformations are represented as `StepRun` s, materialised outputs are represented as artifact instances, and dependencies are captured using `used`, `wasGeneratedBy`, and `wasDerivedFrom` relations. Structured datasets are decomposed into records and cells, while models and evaluation reports are treated as atomic artifacts. Privacy_Provenance (2)

This component is the foundation for later privacy propagation because the privacy proposal defines propagation over executed steps and contributors identified from retrospective provenance relations. It also uses operator types such as `RED`, `TR`, `AGG`, `FUS`, and `LEARN` to determine contributor granularity, minting, and attenuation behaviour. Privacy_Provenance (2)

---

2. Overall tool structure

I suggest naming the prototype:

```
privacyprov
```

or:

```
privacy-aware-mlprovlens
```

The starter package currently uses:

```
privacyprov
```

The overall implementation should eventually have the following components:

```
privacyprov/
│
├── core/
│   ├── enums.py
│   ├── models.py
│   └── graph.py
│
├── instrumentation/
│   ├── context.py
│   └── __init__.py
│
├── privacy/
│   ├── annotation_loader.py          # future
│   ├── operation_policy_loader.py    # future
│   ├── propagation_engine.py         # future
│   ├── operator_rules.py             # future
│   └── ontology.py                   # future
│
├── queries/
│   ├── erasure_impact.py             # future
│   ├── sensitivity_exposure.py       # future
│   ├── transformation_responsibility.py
│   └── consent_compliance.py
│
├── examples/
│   └── asd_workflow_instrumented.py
│
└── outputs/
    ├── asd_instrumented_graph.json
    ├── asd_nodes.csv
    └── asd_relationships.csv
```

For this prompt, I implemented only:

```
core/
instrumentation/
examples/
```

The remaining folders are the next phases.

3. Implemented component 1: Workflow instrumentation layer

The instrumentation layer supports **instrumented pandas/scikit-learn-style workflows**.

The supported workflow style is:

```
load DataFrame
→ register input artifact
→ decorate each pipeline function as a provenance step
→ execute the function normally
→ automatically register output artifacts
→ automatically create StepRun and PROV-style relations
```

This approach is realistic for your supervisor's requirement because it supports a **class of Python scripts**, rather than only the handcrafted ASD walkthrough. It does not claim to analyse arbitrary Python automatically. Instead, it supports explicitly instrumented tabular ML workflows.

---

4. What the instrumentation layer records

For every workflow, it records the following node types:

Node type	Meaning
`Pipeline`	Design-time pipeline container
`PipelineRun`	Concrete execution of the pipeline
`Step`	Design-time operation, with operator type
`StepRun`	Runtime execution of a step
`DatasetArtifact`	DataFrame snapshot
`DataRecordArtifact`	Row-level artifact
`DataCellArtifact`	Optional cell-level artifact
`ModelArtifact`	Trained model
`EvaluationMetric`	Metric such as accuracy or F1

It also records these relationships:

Relationship	Meaning
`executes`	`PipelineRun` executes `Pipeline`; `StepRun` executes `Step`
`contains`	`PipelineRun` contains `StepRun`
`used`	`StepRun` consumed an artifact
`wasGeneratedBy`	Artifact was produced by a `StepRun`
`wasDerivedFrom`	Output artifact was derived from input artifact
`containsDR`	Dataset contains records
`containsRC`	Record contains cells, optional for fine-grained mode

This aligns with MLProvLens, which already integrates fine-grained preprocessing provenance and coarse-grained experiment provenance into a unified graph, and exports that graph into `nodes.csv` and `rels.csv` for interactive querying and visualisation. MLProvLens: Exploring End-to-En...

---

5. Core implementation idea

The main class is:

```
WorkflowContext
```

It manages the provenance graph and provides:

```
register_dataframe(...)
register_model(...)
register_metric(...)
@ctx.step(...)
export_json(...)
export_neo4j_csv(...)
summary()
```

A pipeline function is instrumented like this:

```
@ctx.step(
    name="S5_normalise",
    op=OperatorType.TR,
    inputs=["D6_EnrichedEpisodeData"],
    outputs=["D7_NormalizedEpisodeData"],
    phase="LearningPreparation",
    policy_id="S5_normalise_policy"
)
def normalise(df):
    ...
    return output_df
```

When the function runs, the tool automatically creates:

```
Step
StepRun
used
wasGeneratedBy
wasDerivedFrom
DatasetArtifact
DataRecordArtifact
```

The `policy_id` is important because the next component will load operation policies such as:

```
S5_normalise_policy
S6_select_modelling_features_policy
S8_train_classifier_policy
```

Those policies will later control privacy propagation.

---

6. Example implemented in the package

The starter package includes:

```
privacyprov/examples/asd_workflow_instrumented.py
```

It currently instruments the following steps:

Step	Operation	Operator type
`S5_normalise`	Normalise `HR_res`, `EDA_res`, `ACC_res`	`TR`
`S6_select_modelling_features`	Select modelling attributes and remove identifiers/context fields	`RED`
`S7_split_train_test`	Split modelling dataset into train/test partitions	`RED`
`S8_train_classifier`	Train a decision tree classifier	`LEARN`

This corresponds to the operations we developed manually in the walkthrough.

---

7. Example usage

After unzipping the package, run:

```
cd privacyprov_tool
python -m privacyprov.examples.asd_workflow_instrumented
```

The script produces:

```
outputs/asd_instrumented_graph.json
outputs/asd_nodes.csv
outputs/asd_relationships.csv
```

When I ran it, it generated this graph summary:

```
Nodes:
  Pipeline: 1
  PipelineRun: 1
  DatasetArtifact: 5
  DataRecordArtifact: 16
  Step: 4
  StepRun: 4
  ModelArtifact: 1
  EvaluationMetric: 1

Relationships:
  executes: 5
  containsDR: 16
  contains: 4
  used: 4
  wasGeneratedBy: 5
  wasDerivedFrom: 5
```

This means the workflow instrumentation is already capturing the required base structure.

---

8. Example from the instrumented ASD workflow

The first registered artifact is:

```
ctx.register_dataframe(
    "D6_EnrichedEpisodeData",
    raw,
    role="enriched",
    record_id_col=None
)
```

Then S5 is declared as a transformation:

```
@ctx.step(
    name="S5_normalise",
    op=OperatorType.TR,
    inputs=["D6_EnrichedEpisodeData"],
    outputs=["D7_NormalizedEpisodeData"],
    phase="LearningPreparation",
    policy_id="S5_normalise_policy",
    output_roles={"D7_NormalizedEpisodeData": "normalised"}
)
def normalise(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
```

```
    for src, dst in [
        ("HR_res", "HR_norm"),
        ("EDA_res", "EDA_norm"),
        ("ACC_res", "ACC_norm")
    ]:
        min_v = out[src].min()
        max_v = out[src].max()
        out[dst] = (out[src] - min_v) / (max_v - min_v)

    return out[
        [
            "Id", "window_start", "HR_norm", "EDA_norm", "ACC_norm",
            "chall_behav", "sev", "age_grp", "sex", "clinic_site", "diag"
        ]
    ]
```

Then S6 is declared as a reduction/projection:

```
@ctx.step(
    name="S6_select_modelling_features",
    op=OperatorType.RED,
    inputs=["D7_NormalizedEpisodeData"],
    outputs=["D8_ModellingDataset"],
    phase="LearningPreparation",
    policy_id="S6_select_modelling_features_policy"
)
def select_features(df: pd.DataFrame) -> pd.DataFrame:
    return df[
        [
            "HR_norm", "EDA_norm", "ACC_norm",
            "chall_behav", "sev", "age_grp", "sex"
        ]
    ].copy()
```

This is exactly what we need for later privacy propagation: the graph knows that `D8_ModellingDataset` was produced by a `RED` step and derived from `D7_NormalizedEpisodeData`.

---

9. Why decorators are the right first implementation choice

There are two possible strategies:

Strategy	**Meaning**	**Suitability now**
Decorator-based	User annotates functions using `@ctx.step(...)`	Best first version
Fully automatic parsing	Tool reads arbitrary Python code and infers steps	Too ambitious for now

The decorator-based approach is better for the first prototype because it is deterministic, transparent, and easy to evaluate on 3–5 workflows.

It also supports your supervisor’s requirement: the tool works for a class of Python tabular workflows, provided they are instrumented with the step decorator.

The paper can describe the supported class as:

> instrumented pandas/scikit-learn workflows where pipeline steps are declared using lightweight decorators and mapped to privacy operator types.

---

10. Current scope and limitations of this first component

This first component already captures:

```
pipeline/run metadata
step boundaries
operator type
input/output artifacts
dataset artifacts
record artifacts
model artifacts
evaluation metrics
used / wasGeneratedBy / wasDerivedFrom
Neo4j-style CSV export
JSON export
```

It does **not yet** capture:

```
manual privacy annotations
operation policy dictionaries
privacy propagation
minting
attenuation
role-aware queries
fine-grained cell derivations for every transformation
```

That is intentional. The privacy proposal defines `PrivacyAnnotation` as an object linked to provenance entities with four facets: sensitivity category, usage, erasure, and role. It also allows annotations at different granularities and uses containment for inheritance and summarisation. Privacy_Provenance (2) Those concepts should be implemented in the next phase, after the workflow instrumentation is stable.

---

11. How this connects to the later privacy components

The instrumentation layer produces the graph that the privacy propagation engine will consume.

For example, after running S6, the graph contains:

```
Step: S6_select_modelling_features
Step.operatorType = RED
```

```
StepRun: asd_run_001::002::S6_select_modelling_features

D7_NormalizedEpisodeData --used by--> StepRun
D8_ModellingDataset --wasGeneratedBy--> StepRun
D8_ModellingDataset --wasDerivedFrom--> D7_NormalizedEpisodeData
```

Then the future privacy propagation engine will read:

```
operatorType = RED
policy_id = S6_select_modelling_features_policy
selectedAttributes = [HR_norm, EDA_norm, ACC_norm, chall_behav, sev, age_grp,
```

and infer:

```
C(D8) = {quasi_identifier, biometric, health, behavioural_sensor_data, behavio
U(D8) = {clinical_research}
R(D8) = {delete_on_request, retention_5y}
A(D8) = {DS, CL, AU, DPO}
```

So this first component is not doing privacy reasoning yet, but it creates the exact execution structure needed for privacy reasoning.

---

12. Main code files in the starter package

`privacyprov/core/enums.py`

Defines operator and artifact types:

```python
class OperatorType(str, Enum):
    RED = "RED"
    TR = "TR"
    AGG = "AGG"
    FUS = "FUS"
    LEARN = "LEARN"
    EVAL = "EVAL"
    UNKNOWN = "UNKNOWN"
```

`privacyprov/core/graph.py`

Defines a small in-memory graph:

```
Node
Relationship
ProvenanceGraph
```

It supports:

```
get_or_create_node
add_rel
export_json
export_neo4j_csv
summary
```

`privacyprov/instrumentation/context.py`

Defines:

```
WorkflowContext
```

This is the key file for the first component.

It supports:

```
register_dataframe
register model
register_metric
step decorator
export_json
export_neo4j_csv
```

`privacyprov/examples/asd_workflow_instrumented.py`

Contains the runnable ASD example.

---

13. How to connect this with your existing MetaModel code

Your existing `MetaModel.zip` already has a similar graph-builder idea:

```
MetaNode
MetaRel
ProvGraphBuilder
```

The starter package intentionally follows the same style:

```
Node
Relationship
ProvenanceGraph
```

So there are two options:

Option A — Keep this as a standalone privacy prototype

This is simpler for experimentation and evaluation.

Option B — Integrate it into the existing MetaModel package

This would mean replacing the starter `ProvenanceGraph` with your existing `ProvGraphBuilder`.

For the paper, Option A is enough as a prototype. For the PhD implementation, Option B is cleaner because it unifies the codebase.

---

14. Suggested manuscript wording for this component

You can write:

> The prototype currently supports instrumented Python tabular ML workflows. Each workflow is executed inside a `WorkflowContext`, and each pipeline operation is declared using a lightweight decorator specifying the step name, operator type, input artifacts, output artifacts, and optional policy identifier. During execution, the instrumentation layer automatically records design-time `Step` nodes, runtime `StepRun` nodes, dataset/model/metric artifacts, and PROV-style dependencies (`used`, `wasGeneratedBy`, `wasDerivedFrom`). DataFrame artifacts are additionally decomposed into record-level artifacts, with optional cell-level materialisation. The resulting provenance graph is exported as JSON and as Neo4j-compatible CSV files, providing the structural input required by the later privacy annotation propagation engine.

---

15. Next implementation step

The next step should be **Component 2: Privacy annotation dictionary and ontology loader**.

That component will add:

```
PrivacyAnnotation
SensitivityCategory
UsageConstraint
ErasureConstraint
AccessRole
implies relations
initial dataset/record/attribute annotations
```

Then Component 3 will add the **operation policy dictionary**, and Component 4 will implement the **privacy propagation engine**.

Sources

How the instrumentation layer records Pipeline, pipelineRun, Step, StwpRun and the rest of the nodes, why I can't start from the fine-grained processing provenance graph generated by chapman tool and for the machine learning part, I can import the mlflow2prov provenance output, as well presented in the sourced article
"A_W3C_PROV_Aligned_Metamodel_for_TracingEnd_to_End_Provenance_in_ML_Pipelines"
Examples of these provenance outputs are sourced within the sample_data.zip fodler
Should I start from these provenance outputs, or should I record the nodes and dependencies from scratch as you did in class WorkflowContext

Thought for 6m 33s ›

You are right to question this. For your project, **you should not replace DPDS/Chapman and MLflow2PROV with a new from-scratch recorder**. The better implementation strategy is:

> **Start from the existing provenance outputs generated by DPDS/Chapman for fine-grained data preparation and MLflow2PROV for learning/evaluation, normalize them into your unified meta-model graph, then add the privacy annotation and propagation layer on top.**

The `WorkflowContext` class I sketched earlier is useful as a **fallback/prototype recorder** for simple new Python workflows, but it should not be the main implementation path for your paper because your previous work already built around integrating DPDS and MLflow2PROV. Your meta-model paper explicitly says the implementation is based on a unifying ETL layer that integrates DPDS for fine-grained provenance and MLflow/MLflow2PROV for experimental tracking, materialising the result as a single Neo4j graph queried using Cypher. A_W3C_PROV_Aligned_Metamodel_fo...

---

1. What `WorkflowContext` was doing

The `WorkflowContext` approach records provenance **from scratch** while the Python workflow runs. Conceptually, it creates:

```
Pipeline
PipelineRun
Stage
Step
StepRun
DatasetArtifact
DataRecordArtifact
DataCellArtifact
ModelArtifact
EvaluationMetric
```

and then creates relationships such as:

```
PipelineRun executes Pipeline
PipelineRun contains StepRun
StepRun executes Step
StepRun used Artifact
Artifact wasGeneratedBy StepRun
Artifact wasDerivedFrom Artifact
DatasetArtifact containsDR DataRecordArtifact
DataRecordArtifact containsRC DataCellArtifact
```

This matches the meta-model: your previous paper defines prospective lineage through `Pipeline`, `Stage`, and `Step`, and retrospective lineage through `PipelineRun`, `StepRun`, runtime `Artifact` instances and their specialisations. It also uses standard PROV relations such as `used`, `wasGeneratedBy`, and `wasDerivedFrom` for end-to-end traceability. A_W3C_PROV_Aligned_Metamodel_fo...

So the `WorkflowContext` is a **mini provenance recorder**. It is helpful when no provenance output already exists. But in your case, provenance outputs already exist from the tools you selected.

---

2. Why you should start from DPDS/Chapman and MLflow2PROV outputs

Your previous work is already built on the claim that DPDS/Chapman and MLflow2PROV are complementary:

- DPDS/Chapman provides fine-grained, attribute-level provenance for data preparation and transformation.
- MLflow2PROV captures run-level, model-level, parameter-level, metric-level, and artifact-level experiment provenance.
- Your meta-model integrates both into a lifecycle-wide unified representation. A_W3C_PROV_Aligned_Metamodel_fo...

This is important because your new privacy work should be framed as:

> **A privacy-aware extension of an existing end-to-end ML provenance integration framework**, not as a new provenance capture system.

That is stronger academically. It lets you say:

1. provenance capture is handled by existing tools;
2. your contribution is the **privacy ontology**, **privacy annotation loading**, **operator-aware propagation**, and **privacy auditing queries**;
3. the privacy layer is generic because it operates over the unified meta-model graph, not over a single handcrafted script.

---

3. The correct architecture: provenance acquisition, not only instrumentation

I suggest renaming the first component from:

```
Workflow instrumentation layer
```

to:

```
Provenance acquisition and normalization layer
```

This layer should support **three input modes**:

Mode	Purpose	Status
DPDS/Chapman importer	Import fine-grained preprocessing provenance	Main path
MLflow2PROV importer	Import learning, model, metric, and run provenance	Main path
WorkflowContext recorder	Capture simple pandas/scikit-learn workflows when DPDS/MLflow2PROV are unavailable	Optional/fallback

So your implementation structure should become:

```
privacyprov/
│
├── core/
│   ├── graph.py
│   ├── models.py
│   └── enums.py
│
├── importers/
│   ├── base_importer.py
│   ├── dpds_importer.py
│   ├── mlflow2prov_importer.py
│   ├── unified_csv_importer.py
│   └── sample_data_importer.py
│
├── integration/
│   ├── normalizer.py
│   ├── cross_stage_linker.py
│   └── validator.py
│
├── instrumentation/
│   └── workflow_context.py   # optional fallback, not main path
│
├── privacy/
│   ├── annotation_loader.py
│   ├── policy_loader.py
│   ├── ontology.py
│   ├── propagation_engine.py
│   └── operator_rules.py
│
└── queries/
    ├── erasure_impact.py
    ├── sensitivity_exposure.py
    ├── transformation_responsibility.py
    └── consent_compliance.py
```

The important shift is that `WorkflowContext` moves from being the main recorder to being an optional utility. The main path becomes:

```
DPDS output + MLflow2PROV output
```